NIT2112
Object Oriented Programming

Session 07

AI Copilot and Prompt Engineering

VICTORIA UNIVERSITY
MELBOURNE AUSTRALIA

Acknowledgement of Country

Victoria University acknowledges, recognises and respects the Ancestors, Elders and families of the Bunurong/Boonwurrung, Wadawurrung and Wurundjeri/Woiwurrung of the Kulin who are the traditional owners of University land in Victoria, the Gadigal and Guring-gai of the Eora Nation who are the traditional owners of University land in Sydney, and the Yugara/YUgarapul people and Turrbal people living in Meanjin (Brisbane).

Session Outline

Understanding AI Copilots

- Generative AI, LLMs & Tools
- Benefits & Drawbacks

Prompt Engineering

- Definition & Importance
- Key Elements & Best Practices

AI Copilot

The Hottest New Programming Language

- "The Hottest New Programming Language is English." - Andrej Karpathy
- "Copilot has dramatically accelerated my coding, it's hard to imagine going back to "manual coding". Still learning to use it but it already writes ~80% of my code, ~80% accuracy. I don't even really code, I prompt. & edit."
- Andrej Karpathy: a Slovak-Canadian computer scientist who served as the director of artificial intelligence and Autopilot Vision at Tesla. He co-founded and formerly worked at OpenAI, where he specialized in deep learning and computer vision.

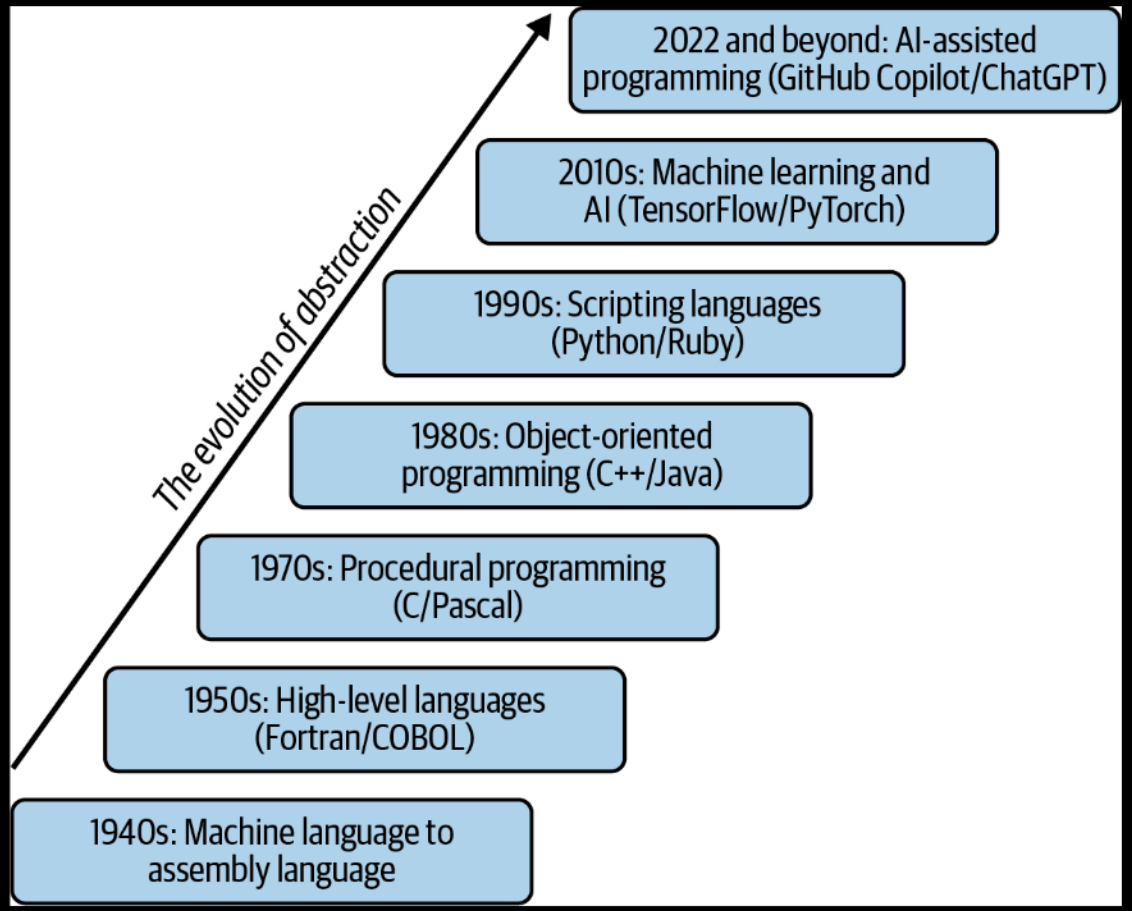
* How do you interpret this quote in the context of software development?

Andrej Karpathy

Image from Andrej Karpathy, 2025, https://karpathy.ai/

Abstraction: Making Development Easier

- **Abstraction** simplifies systems for developers.
- Tedious details are handled in the background.
- Focus shifts to **core functionality** and **problem-solving**.
- Abstraction drives **innovation** (Internet, Cloud, Mobile, AI)
- Evolution of complexity increases, developers focus on **high level logic**.



Taulli, Tom. *AI-Assisted Programming: Better Planning, Coding, Testing, and Deployment."* O'Reilly Media, Inc.", 2024.

Early Days: Machine Language and Assembly Language

- Machine Language (1940s): 0s and 1s - Direct hardware manipulation.
- Assembly Language: **Symbolic representation**. Alphanumeric instructions for easier coding.
- **Mnemonics and labels**: Reduced errors and improved readability.
- **One-to-one correspondence** with machine instructions.

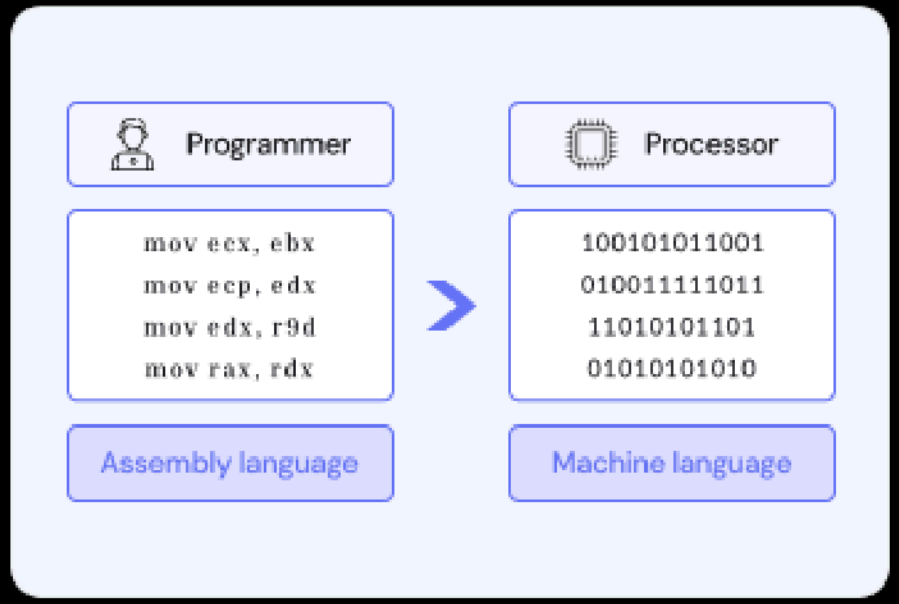


Image from Limeup, 2025, https://limeup.io/glossary/assembly-language/

High-Level Languages: A Revolution

- Fortran and COBOL (1950s): Plain English-like commands.
- Examples: DISPLAY, READ, WRITE, IF/THEN/ELSE
- **Executable instructions**: Compilers translated code into machine.
- **Increased accessibility**: More readable code for non-technical users.
- **Portability**: Allowed for platform independence.

```
! ------------------------------------------------------
! Compute the area of a triangle using Heron's formula
! ------------------------------------------------------

PROGRAM  HeronFormula
   IMPLICIT  NONE

   REAL     :: a, b, c             ! three sides
   REAL     :: s                   ! half of perimeter
   REAL     :: Area                ! triangle area
   LOGICAL  :: Cond_1, Cond_2      ! two logical conditions

   READ(*,*)  a, b, c

   WRITE(*,*)  "a = ", a
   WRITE(*,*)  "b = ", b
   WRITE(*,*)  "c = ", c
   WRITE(*,*)

   Cond_1 = (a > 0.) .AND. (b > 0.) .AND. (c > 0.0)
   Cond_2 = (a + b > c) .AND. (a + c > b) .AND. (b + c > a)
   IF (Cond_1 .AND. Cond_2) THEN
      s    = (a + b + c) / 2.0
      Area = SQRT(s * (s - a) * (s - b) * (s - c))
      WRITE(*,*) "Triangle area = ", Area
   ELSE
      WRITE(*,*) "ERROR: this is not a triangle!"
   END IF

END PROGRAM  HeronFormula
```

Fortran Language

Image from Declan Valters, 2025, https://ourcodingclub.github.io/tutorials/fortran-intro/

Procedural Programming: Organizing Tasks

- C and Pascal (1970s): Introduced functions (procedures).
- **Modular Design**: Complex tasks were broken down into reusable functions.
- **Reduced redundancy**: Improved code reusability and maintainability.
- **Top-down approach**: Simplified the management of large software projects.

C Language: Procedural Programming

Object-Oriented Programming: Modelling the World

- C++ (1985) and Java (1995): Model real-world entities as classes and objects.
- **Encapsulation** of data and behaviour: Improved security and organization.
- Reusability through **inheritance**: Promoted modularity and intuitive problem-solving.
- **Polymorphism** enabled flexible code design.

```
Welcom.java > ...
1   public class Welcom {
        Run | Debug
2       public static void main(String[] args) {
3           System.out.println("Hello World!");
4       }
5   }
```

Java: Object Oriented Programming

Scripting Languages: Simplifying Tasks

- Python, Ruby, JavaScript (1990s): Abstract lower-level tasks.
- **Rapid development**: Extensive libraries and built-in data structures.
- **Dynamic typing**: Simplified common programming tasks.
- **Interpreted languages**: Reduced the amount of code needed for web development.

```python
from flask import Flask

app = Flask(__name__)

@app.route("/")
def hello_world():
    return "<p>Hello, World!</p>"
```

Python: A minimal web server using Python Flask

Machine Learning and AI: Automating Insights

- TensorFlow and PyTorch (2010s): Abstract complex mathematical details.
- **Neural network design**: Focus on model architecture and training processes.
- **Data-driven insights**: Enabled developers to build intelligent systems with less math expertise.
- **Automated** feature extraction and pattern recognition.

```python
from torch import nn

# We define a CNN with 3 convolutional layers.
# Each convolution is followed by a ReLU.
# At the end, we perform an average pooling.
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 16, kernel_size=3, stride=2, padding=1)
        self.conv2 = nn.Conv2d(16, 16, kernel_size=3, stride=2, padding=1)
        self.conv3 = nn.Conv2d(16, 10, kernel_size=3, stride=2, padding=1)

    def forward(self, xb):
        xb = xb.view(-1, 1, 28, 28)
        xb = F.relu(self.conv1(xb))
        xb = F.relu(self.conv2(xb))
        xb = F.relu(self.conv3(xb))
        xb = F.avg_pool2d(xb, 4)
        return xb.view(-1, xb.size(1))
```

PyTorch: A three-layer convolutional neural network

AI-Assisted Programming: Your Coding Assistant

- GPT-4 and other LLMs (2020s): Generate code on command - Natural language interface.
- **Context-aware suggestions**: Automate repetitive tasks and suggest solutions.
- **Accelerated development cycles**: Potential to significantly increase developer productivity.
- **Lowered barrier** to entry for new programmers.

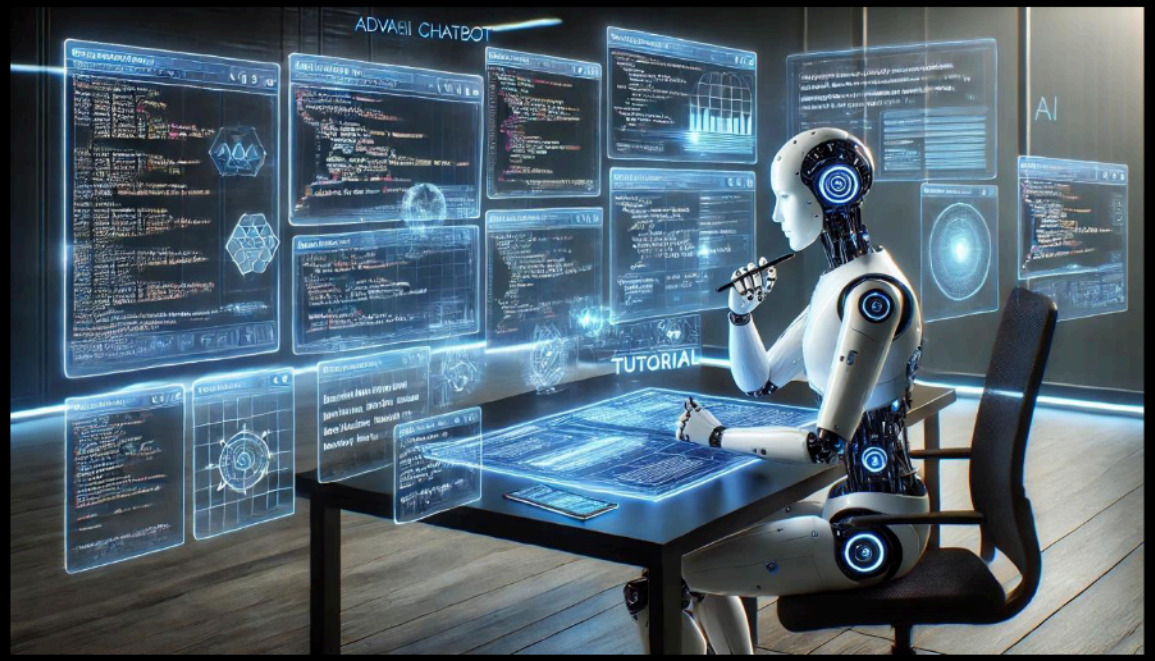


Image from Murtaza Ali, Jan 16 2025, https://towardsdatascience.com/the-death-of-human-written-code-tutorials-in-the-chatgpt-era-or-not-9a437a58a0b2/

Think-Pair-Share: Your Experience

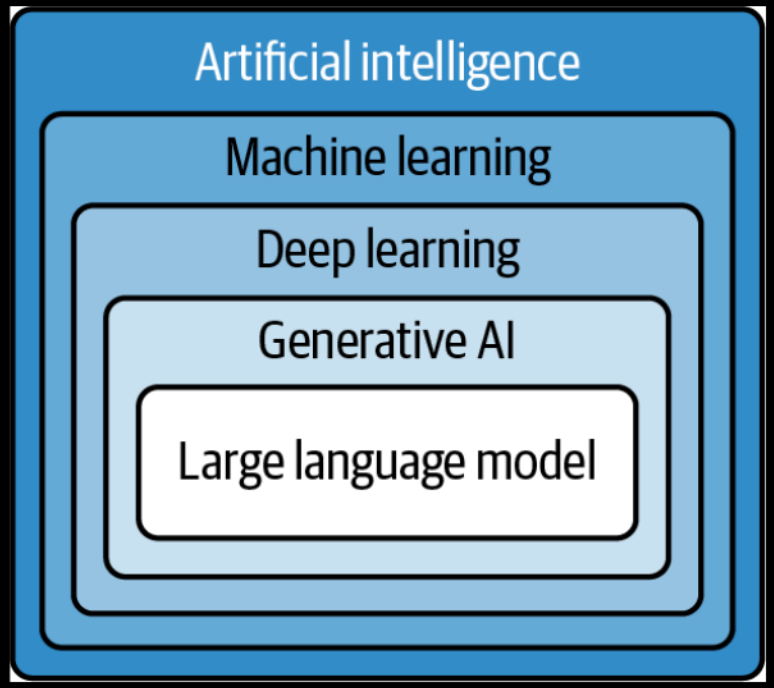
Question:

- What programming language abstraction have you found most impactful in your experience, and why?
- How do you see AI-assisted programming changing the landscape of software development?

* Instructions: Take 2 minutes to think individually, then discuss with a partner for 3 minutes.

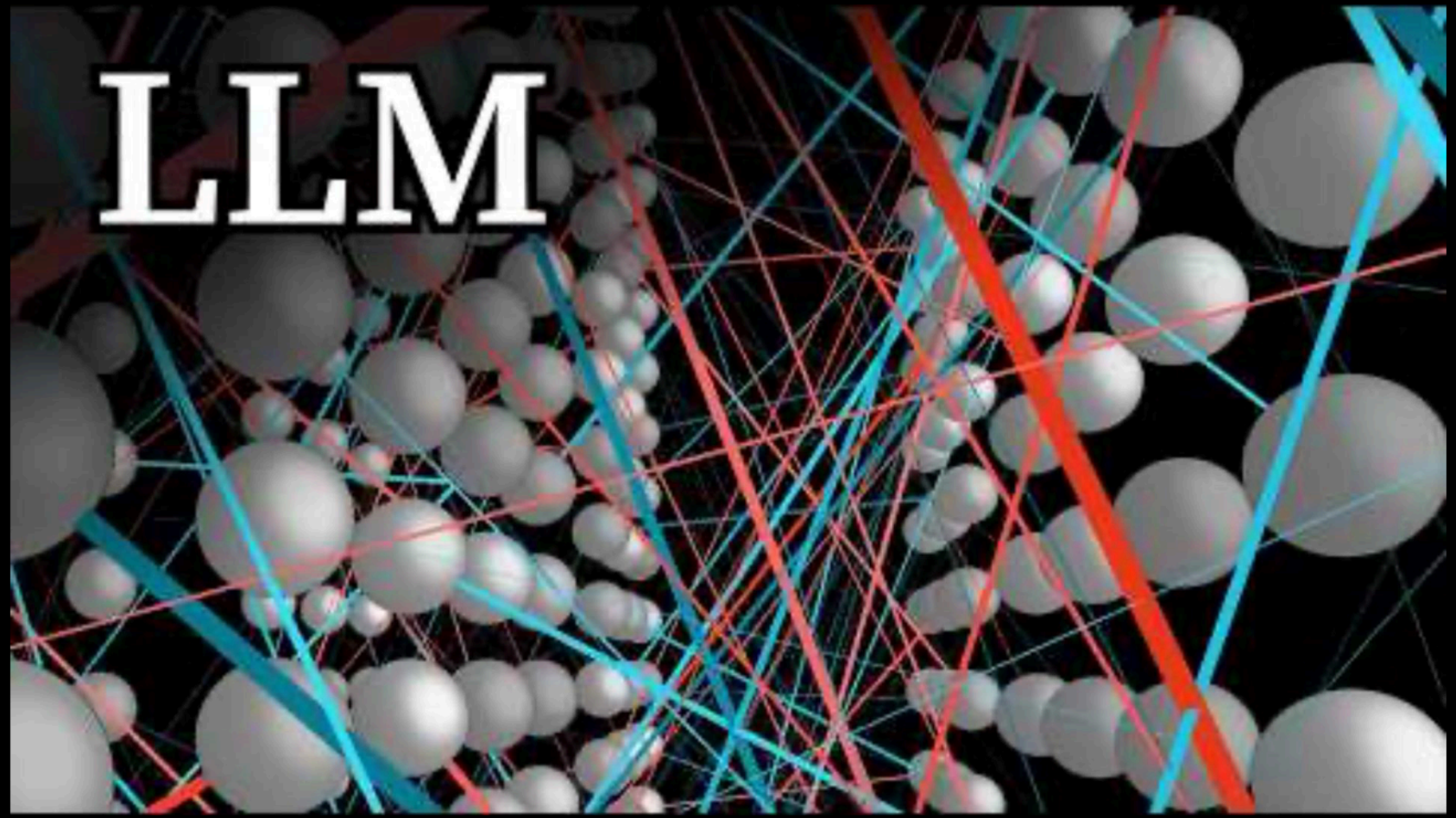
The AI Family Tree

- **AI**: Systems mimicking human intelligence - Broadest scope.
- **Machine Learning**: Systems learning from data without explicit instructions - Algorithms and models.
- **Deep Learning**: A subset of ML using neural networks with multiple layers - Complex architectures.
- **Generative AI:** Creates new data resembling training data - Novel output generation.
- **Large Language Models (LLMs)**: Powerful models (GPT-4, Gemini, Claude) generating human-like text - Natural language understanding.



Taulli, Tom. *AI-Assisted Programming: Better Planning, Coding, Testing, and Deployment*." O'Reilly Media, Inc.", 2024.

Transformer and LLM



Large Language Models explained briefly, 3Blue1Brown, Nov 21, 2024, https://youtu.be/LPZh9BOjkQs?si=W7_a2J5aU5kEy2X-

Generative AI: More Than Just Text

- **Multimodal capabilities**: Create images, audio, and video - Integrated understanding.
- Expanded creative potential: Applications in design, music, and film.

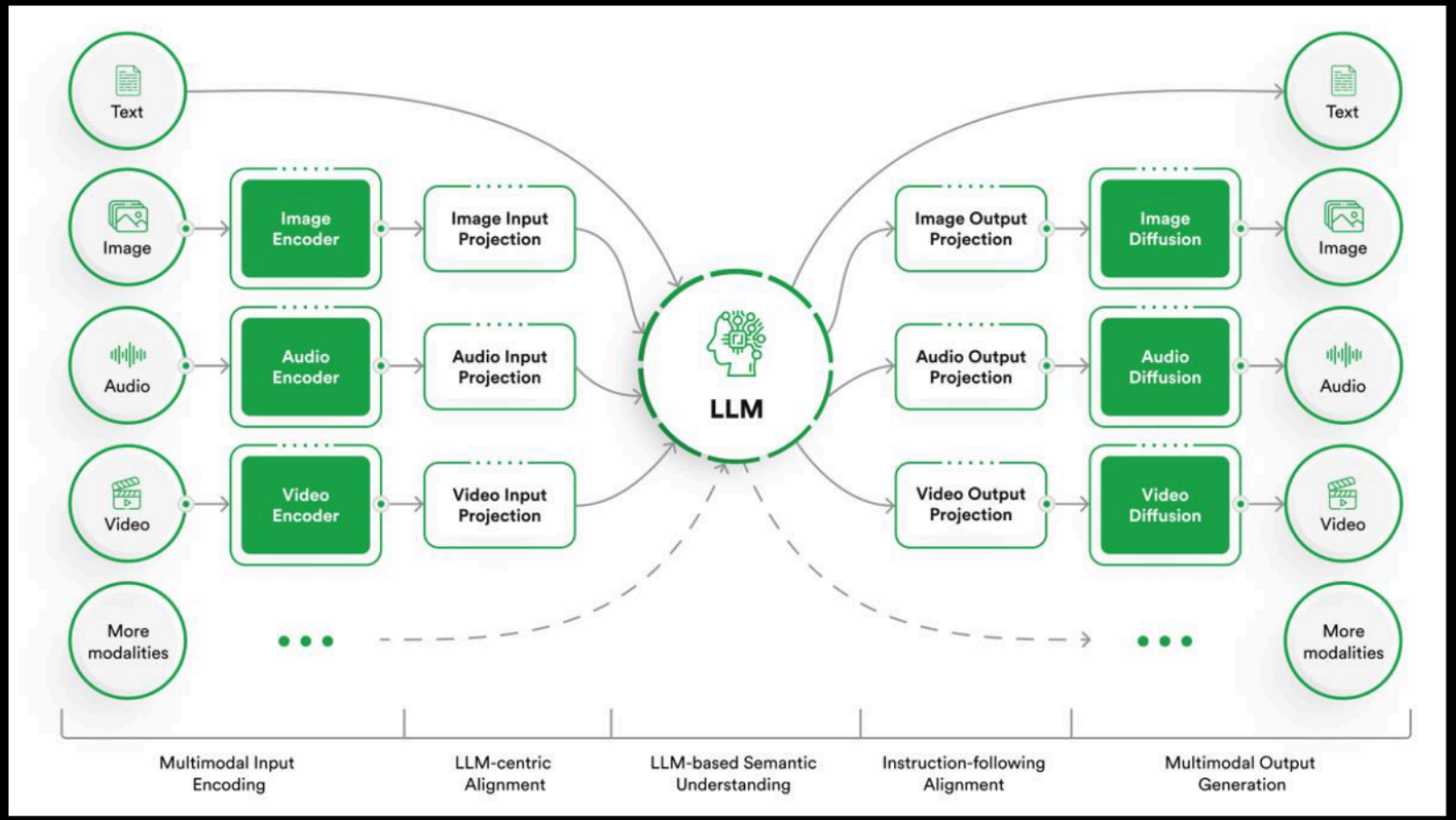


Image from Rinf.Tech, 2023, https://www.rinf.tech/how-multimodal-ai-is-transforming-our-interaction-with-technology/

AI Assisted Programming Tools: Your Reliable Copilot

- **Increased efficiency**: Enhance developer capabilities.
- **Strategic contributions**: Focus on advanced problem-solving and innovation.
- **Streamlined workflows**: Reduce monotonous tasks and complex code details.
- **Faster learning** of new languages and frameworks.

Image by Jason Blair, FAA Designated Pilot Examiner (DPE), May 3, 2025,
https://medium.com/faa/the-dangers-of-overreliance-on-automation-5b7afb56ebdc

Benefits of AI Copilots (1/3): Speed and Efficiency

- **Faster Code Writing**: Auto-completing lines, functions, and even complex algorithms significantly speeds up typing.
- **Boilerplate Reduction**: Generates repetitive code structures (e.g., class definitions, getters/setters, loop structures) quickly.
- **Reduced Context Switching**: Finding solutions or remembering syntax without leaving the IDE keeps you focused.

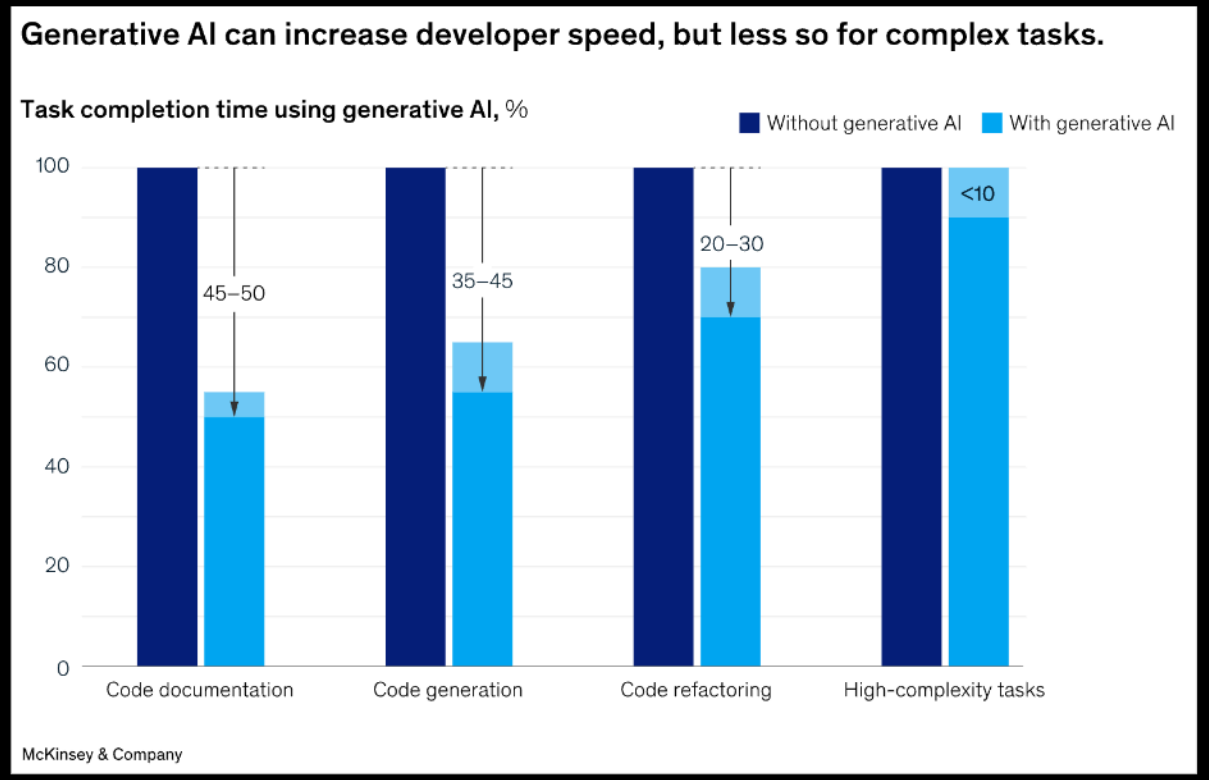
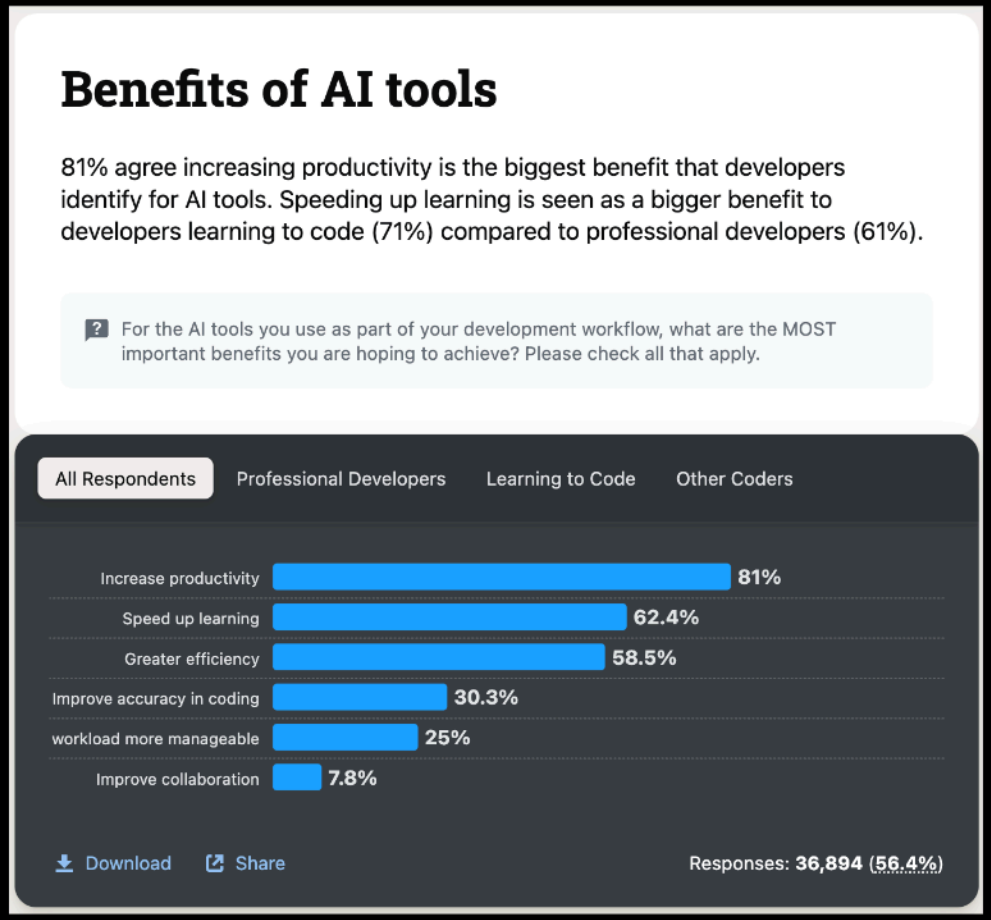


Image from McKinsey & Company, June 27, 2023, https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/unleashing-developer-productivity-with-generative-ai

Benefits of AI Copilots (2/3): Learning and Exploration

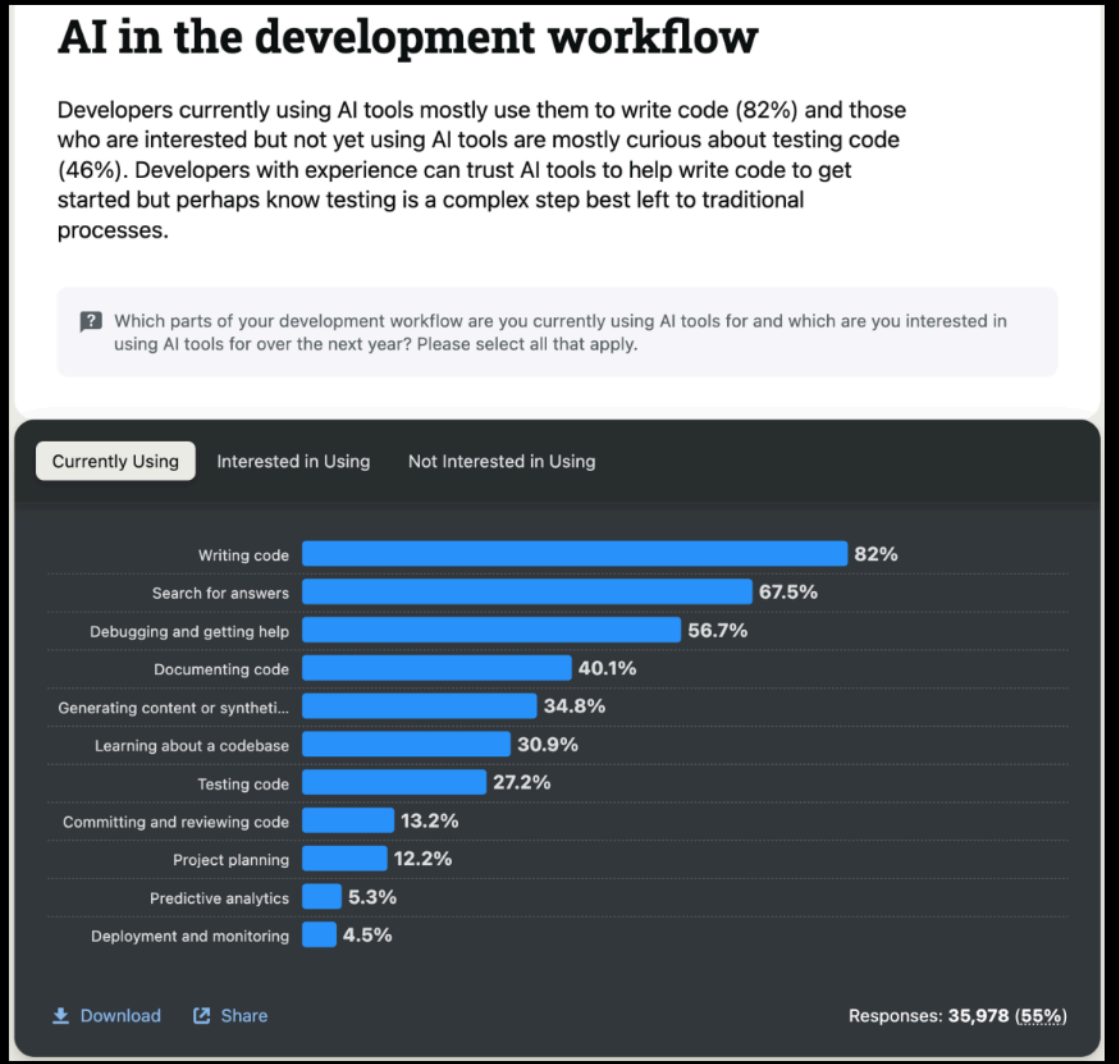
- **Discovering APIs and Libraries**: Suggests functions or methods you might not know exist.
- **Learning New Patterns**: Exposes you to different ways of solving a problem or structuring code (though not always the best way!).
- **Understanding Unfamiliar Code**: Can sometimes help explain or document code snippets (using chat features or generating comments).



2024 Developer Survey, Image from 2024 Stack Exchange Inc, June 27, 2023, https://survey.stackoverflow.co/2024/ai/

Benefits of AI Copilots (3/3): Reducing Tedium & Assisting Debugging

- **Automating Repetitive Tasks**: Writing unit tests, generating documentation strings (docstrings), formatting data.
- **Suggesting Bug Fixes**: Some Copilots can analyze code and suggest potential fixes for common errors.
- **Refactoring Assistance**: Can help restructure or rewrite code based on instructions (e.g., "convert this function to a class method").



2024 Developer Survey, Image from 2024 Stack Exchange Inc, June 27, 2023, https://survey.stackoverflow.co/2024/ai/

Example: Boilerplate Reduction (Python OOP)

- Prompt:

Python class 'Dog' with attributes name (string), breed (string).
Include __init__ constructor.
Include a method 'bark' that prints "Woof!".

- Potential AI Output:

```python
class Dog:
    def __init__(self, name: str, breed: str):
        self.name = name
        self.breed = breed

    def bark(self):
        print("Woof!")
```

Group Discussion

Question:

- How do you currently utilize AI Copilot to assist in programming?
- What challenges have you encountered while using AI Copilot?

* Instructions: Take 2 minutes to think individually, then discuss with a partner for 3 minutes.

Drawbacks and Risks (1/3): Accuracy and Correctness

- **Code Can Be Incorrect**: AI doesn't 'understand' code; it predicts likely sequences. Suggestions can contain subtle or obvious bugs.
- **Hallucinations**: May generate plausible-looking but logically flawed or nonsensical code.
- **Security Vulnerabilities**: Generated code might introduce security risks (e.g., SQL injection, improper handling of credentials) if not carefully reviewed.
- **Outdated Information**: May suggest code based on deprecated libraries or old practices found in its training data.

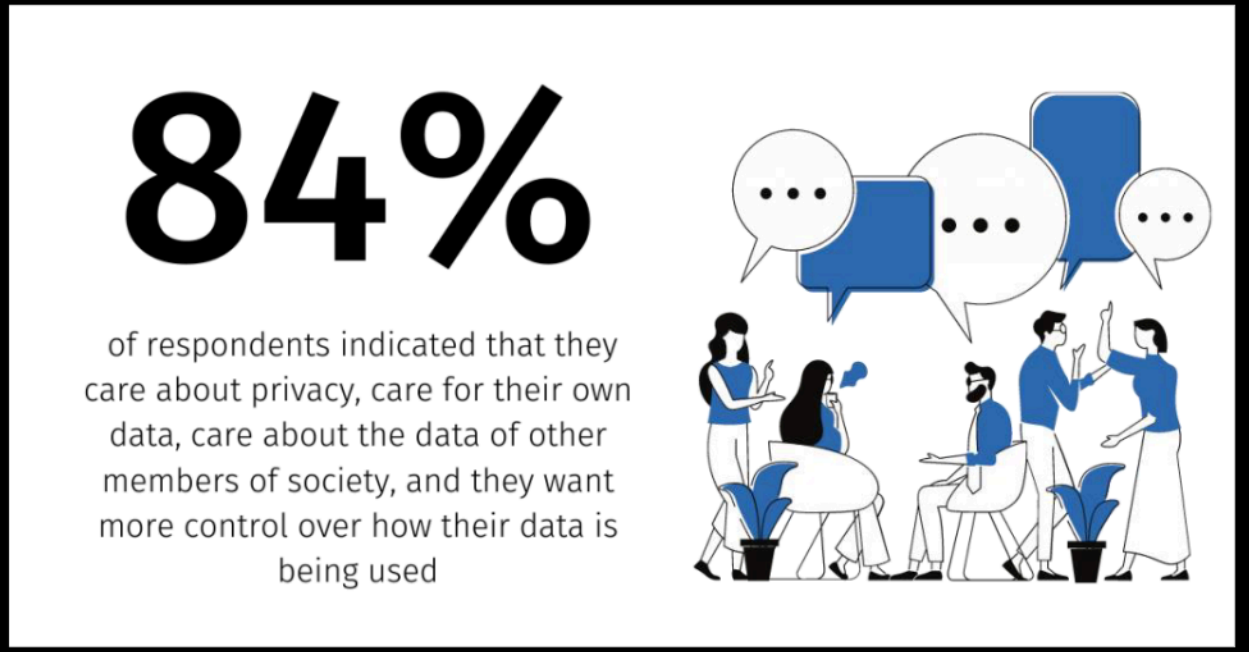
* In this example, Sora fails to model the chair as a rigid object, leading to inaccurate physical interactions.

Archeologists discover a generic plastic chair in the desert, excavating and dusting it with great care.

Chair Found by Archaeologists - OpenAI Sora, SORA-OpenAI, Mar 3, 2024, https://youtu.be/GxBXtQeE_mA?si=RPUsshJlUmx_KyQm

Drawbacks and Risks (2/3): Training Data Bias, Fairness, Privacy

- **Reflecting Training Data Bias**: AI models learn from the data they are trained on. If the training data (e.g., public code repositories) contains biases (e.g., non-inclusive language, inefficient algorithms for certain groups), the AI may replicate and even amplify them.
- **Lack of Diverse Solutions**: May disproportionately suggest common or popular solutions, potentially overlooking more optimal or creative niche solutions.
- **Data privacy:** Concerns about data usage for training purposes, especially for those cloud-based AI tools.



Cisco 2019 Consumer Privacy Survey (Report), Cisco, 2019. Access: https://www.cisco.com/c/dam/global/en_uk/products/collateral/security/cybersecurity-series-2019-cps.pdf

Drawbacks and Risks (3/3): Over-reliance and Learning Impact

- **Reduced Understanding**: Relying too heavily on generated code without understanding how it works can hinder learning and problem-solving skills.
- **Deskilling**: Potential for erosion of fundamental coding skills if developers become too dependent.
- **Intellectual Property (IP) and Licensing**: The ownership and licensing of AI-generated code can be complex. Training data might include code with various licenses; generated code could inadvertently resemble proprietary or copylefted code snippets. (This is a legally complex and evolving area!)
- **Cost**: Most powerful Copilot tools are subscription-based.

Image from Chat GPT Is Eating the World, February 19, 2025, https://chatgptiseatingtheworld.com/2025/02/19/status-of-all-39-copyright-lawsuits-v-ai-feb-18-2025-judge-bibas-rejects-fair-use-in-ai-training-in-stunning-reversal/

Ethical Considerations

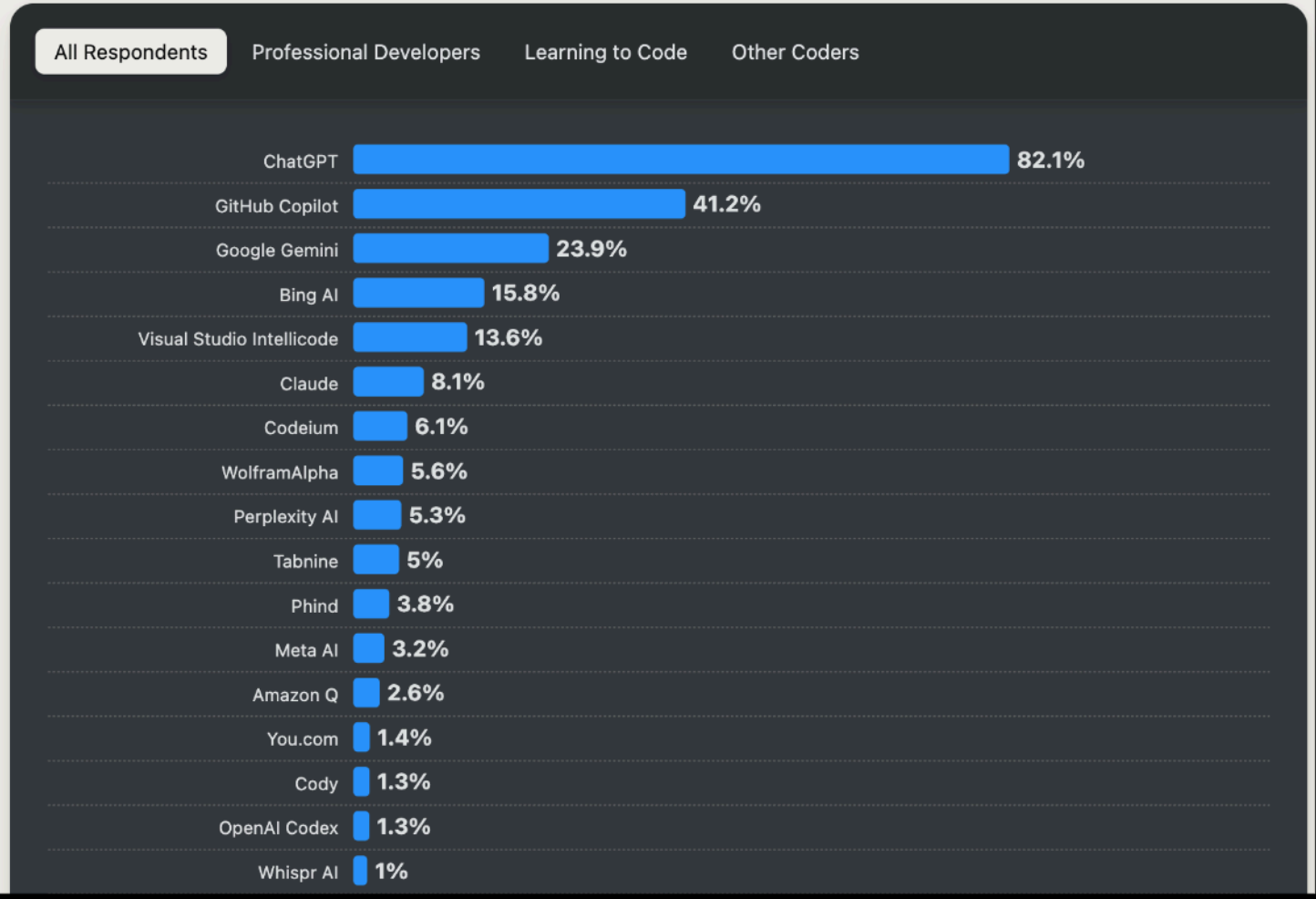
- **Accountability**: Who is responsible if AI-generated code causes harm or fails? (The developer who accepted it!)
- **Job Displacement**: Concerns about AI automating programming tasks. (Augmentation vs. replacement).
- **Transparency**: Understanding how AI models make decisions (often difficult - "black box" problem).
- **Fairness & Bias**: Ensuring AI tools don't perpetuate societal biases.
- **Environmental Impact**: Training large models requires significant computational resources and energy.

Current Landscape of AI-Assisted Programming Tools

AI Search and Developer Tools

ChatGPT is used by twice as many developers as its next closest alternative, GitHub Copilot. ChatGPT has a popular free option that developers observably like.

Which **AI-powered search and developer tools** did you use regularly over the past year, and which do you want to work with over the next year? Select all that apply.

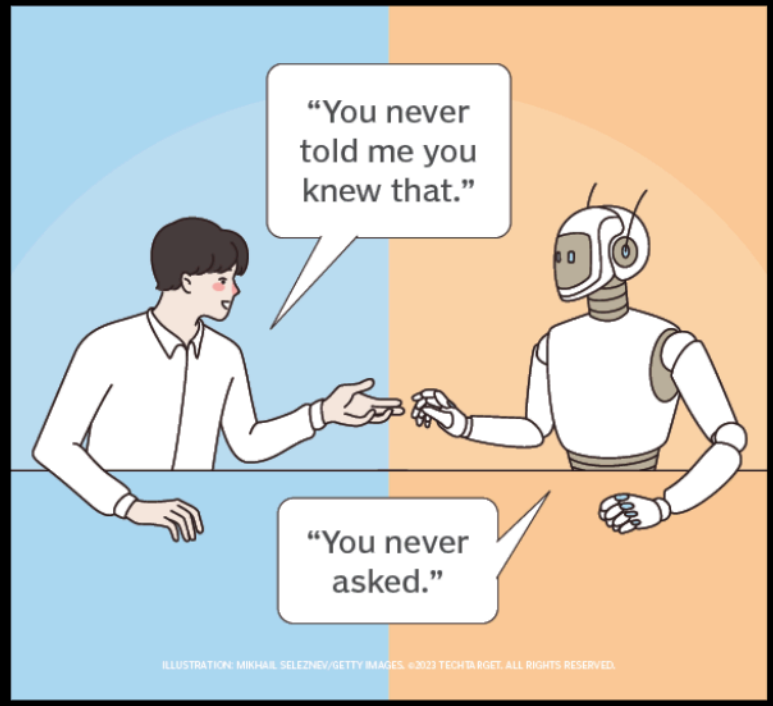


2024 Developer Survey, Image from 2024 Stack Exchange Inc, June 27, 2023, https://survey.stackoverflow.co/2024/ai/

Prompt Engineering

What is Prompt Engineering?

- Definition: The process of **designing**, **refining**, and **optimizing** input prompts to effectively guide Generative AI models (like LLMs) towards producing desired outputs.
- It's both an **art** and a **science**. Requires:
  - Understanding the AI's capabilities and limitations.
  - Clarity of thought about the desired outcome.
  - Experimentation and iteration.
- Think of it like communicating with a very powerful, knowledgeable, but extremely literal assistant who lacks common sense and context unless explicitly provided.



By Robert Sheldon & Kinza Yasar, Nov 13, 2024,
https://www.techtarget.com/searchenterpriseai/definition/AI-prompt

Why is Prompt Engineering Important?

- **Controls Output Quality**: Well-crafted prompts lead to more accurate, relevant, and useful responses.
- **Saves Time & Effort**: Reduces the need for multiple attempts and extensive editing of AI output.
- **Unlocks AI Potential**: Enables you to leverage the AI for more complex and nuanced tasks.
- **Mitigates Risks**: Can help steer the AI away from generating incorrect, biased, or insecure code (e.g., by specifying constraints).

Analogy: Giving Directions

- Poor Prompt (Vague Directions):

How do I get downtown?

- Potential AI Output:

Sure, I can help with that! Could you let me know:

1. **Which city you're in?**
2. **Where you're starting from (address or landmark)?**
3. **Your preferred method of travel** (e.g., walking, driving, public transit)?

Once I have that, I'll give you the best route! 🚉

- Good Prompt (Specific Directions):

Provide driving directions from Victoria University Footscray to Melbourne City Hall using the fastest route, avoiding toll roads. Assume I am leaving now (around 2 PM on a weekday).

- Potential AI Output:

To drive from Victoria University Footscray Campus to Melbourne City Hall without using toll roads, follow these directions:

1. **Departing Victoria University Footscray Campus:**
   - Head east on Ballarat Road (State Route 39).
2. **Continue on Ballarat Road:**
   - Proceed eastward; Ballarat Road transitions into Smithfield Road.
3. **Turn Right onto Epsom Road:**
   - After approximately 2 km, turn right onto Epsom Road.
4. **Continue onto Langs Road:**

Challenges in Prompt Engineering

- **Ambiguity**: Natural language is often ambiguous; AI might interpret your prompt differently than intended.
- **Context Limitation**: AI models have limits on how much context they can remember (context window size).
- **Sensitivity to Phrasing**: Minor changes in wording can sometimes lead to drastically different outputs.
- **Bias Amplification**: Poorly phrased prompts can inadvertently trigger or amplify biases in the model.
- **Creativity vs. Specificity Trade-off**: Sometimes being too specific stifles creative or unexpected (but useful) outputs. Finding the right balance.
- **Knowing What's Possible**: Understanding the AI's capabilities to ask for achievable things.

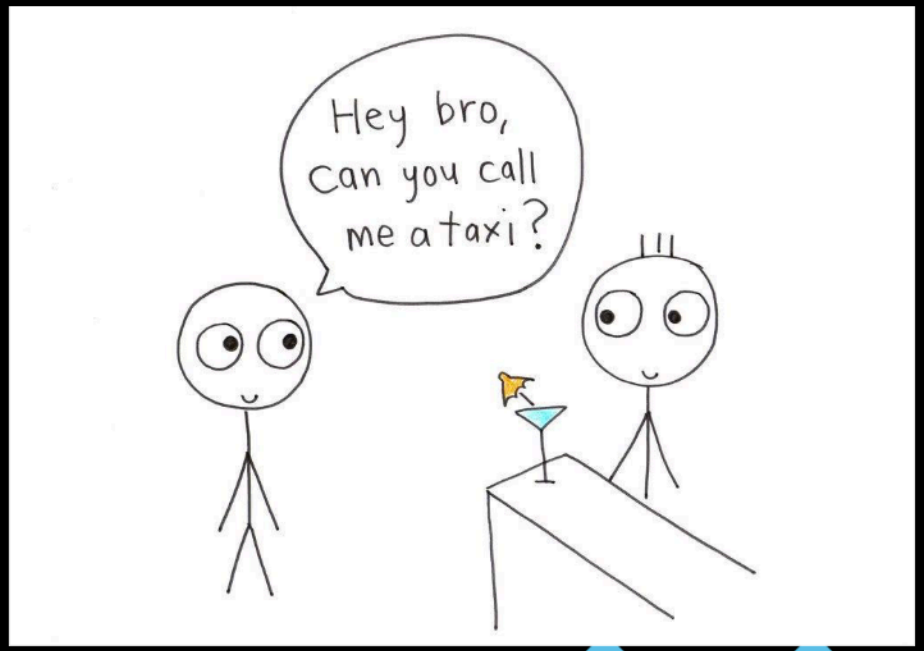


Image from Pedro Borges, May 4, 2023, https://medium.com/@pedroborgespc/spotting-ambiguity-how-people-and-ai-differ-in-understanding-confusion-phrases-9d068832552c

Interactive Question: Clear Communication

Question:

- Think about when you ask a colleague or friend for help with a coding problem. What information do you need to provide them to get useful help quickly? How is this similar to prompting an AI?

* Instructions: Take 2 minutes to think individually, then discuss with a partner for 3 minutes.

Key Elements of a Good Prompt (1/6): Clarity and Specificity

- **Be Precise**: Clearly state the task, the subject, and the expected outcome. Avoid vague language.
- **Define Scope**: Be specific about the boundaries of the request. What should be included? What should be excluded?
- **Use Unambiguous Language**: Choose words carefully to minimize potential misinterpretations.

- Poor Prompt:

> *Fix my code.*

- Good Prompt:

> *Review the following Python function calculate_discount.*
> *It's throwing a TypeError when the item_price is None.*
> *Suggest a fix to handle None values gracefully by returning 0 discount.*

Key Elements of a Good Prompt (2/6): Context

- **Provide Background**: Include relevant information the AI needs to understand the request.
  - Existing code snippets.
  - Error messages.
  - The programming language or framework being used.
  - The overall goal of the code.
- **Use Delimiters**: Clearly separate instructions from context (e.g., using code, [CONTEXT], ###).
- Example:

Given the following Python class
```
class User:
    def __init__(self, username):
    self.username = username
```
Add a method set_password(self, password) that hashes the password using bcrypt before storing it in an attribute self.__hashed_password.

Key Elements of a Good Prompt (3/6): Persona / Role-Playing

- **Assign a Role**: Instruct the AI to act as a specific persona. This influences its tone, style, and focus.
- Effect: Can significantly shape the output to match desired expertise level and perspective.
- Examples:
  - "Act as a senior Python developer specializing in secure coding practices..."
  - "Explain this concept as if you were teaching a beginner programmer..."
  - "Respond as a meticulous code reviewer, focusing on potential bugs and edge cases..."

Key Elements of a Good Prompt (4/6): Desired Output Format

- **Specify Structure**: Tell the AI how you want the output formatted.
- Benefit: Ensures the output is readily usable for your specific needs.
- Examples:
  - "Provide the output as a JSON object."
  - "List the steps in a numbered list."
  - "Generate only the Python function, without any explanatory text."
  - "Write Python code with type hints and docstrings."
  - "Explain the differences in a table format."

Key Elements of a Good Prompt (5/6): Constraints and Negative Constraints

- **Set Boundaries**: Define limitations or requirements.
  - "Use only standard Python libraries."
  - "The function should not exceed 20 lines of code."
  - "Ensure the solution has O(n log n) time complexity."
- Specify **What Not To Do** (Negative Constraints):
  - "Do not use recursion."
  - "Avoid using external APIs."
  - "Don't include print statements in the function."
- Purpose: Guides the AI towards acceptable solutions and avoids unwanted approaches.

Key Elements of a Good Prompt (6/6): Examples (Few-Shot Prompting)

- **Provide Examples**: Include a few examples of the input/output pattern you want the AI to follow. This is known as "Few-Shot Prompting".
- How it helps: Shows the AI the desired style, format, and logic more effectively than just describing it.
- Example (Code Generation Style):

```
Generate Python docstrings in the Google style. Example:

Input function:
def add(a, b):
 return a + b

Output docstring:
"""Adds two numbers together.

Args:
 a: The first number.
 b: The second number.

Returns:
 The sum of a and b.
"""
Now, generate a docstring for this function:

def greet(name):
 print(f"Hello, {name}!")
```

Example: Poor vs. Good Prompt

- Task: Create a simple Rectangle class.

- Poor Prompt:

> *Write a Python class for a rectangle.*

- Potential AI Output:

```python
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        """Returns the area of the rectangle."""
        return self.width * self.height

    def perimeter(self):
        """Returns the perimeter of the rectangle."""
        return 2 * (self.width + self.height)

    def __str__(self):
        return f"Rectangle(width={self.width}, height={self.height})"

    def is_square(self):
        """Returns True if the rectangle is a square, otherwise False."""
        return self.width == self.height
```

Example: Poor vs. Good Prompt

- Task: Create a simple Rectangle class.
- Good Prompt:

Act as a senior Python developer. Write a Python class named 'Rectangle'.

- *It should have private attributes for '__width' and '__height'.*
- *Include an __init__ method that accepts width and height, raising a ValueError if either is negative.*
- *Implement public property methods (getter and setter) for width and height.*
- *The setters should also validate that the value is non-negative.*
- *Include a method 'calculate_area()' that returns the area.*
- *Include a method '__str__' that returns a string representation like 'Rectangle(width=W, height=H)'.*
- *Add clear docstrings for the class and all methods.*

- Potential AI Output:

```python
class Rectangle:
    """
    A class to represent a rectangle with width and height.

    Attributes:
        __width (float): The width of the rectangle (private).
        __height (float): The height of the rectangle (private).
    """

    def __init__(self, width, height):
        """
        Initialize a Rectangle instance.

        Args:
            width (float): The width of the rectangle.
            height (float): The height of the rectangle.

        Raises:
            ValueError: If width or height is negative.
        """
        if width < 0 or height < 0:
            raise ValueError("Width and height must be non-negat
        self.__width = width
        self.__height = height

    @property
    def width(self):
        """Get the width of the rectangle."""
        return self.__width

    @width.setter
    def width(self, value):
        """
        Set the width of the rectangle.

        Args:
            value (float): The new width value.

        Raises:
            ValueError: If value is negative.
        """
        if value < 0:
            raise ValueError("Width must be non-negative.")
        self.__width = value

    @property
    def height(self):
        """Get the height of the rectangle."""
        return self.__height

    @height.setter
    def height(self, value):
        """
        Set the height of the rectangle.

        Args:
            value (float): The new height value.

        Raises:
            ValueError: If value is negative.
        """
        if value < 0:
            raise ValueError("Height must be non-negative.")
        self.__height = value

    def calculate_area(self):
        """
        Calculate and return the area of the rectangle.

        Returns:
            float: The area of the rectangle.
        """
        return self.__width * self.__height

    def __str__(self):
        """
        Return a string representation of the rectangle.

        Returns:
            str: A string in the format 'Rectangle(width=W, height=H)'.
        """
        return f"Rectangle(width={self.__width}, height={self.__height})"
```

Best Practices for Prompt Engineering (1/5): Iterative Refinement

- **Don't Expect Perfection First**: Your first prompt might not yield the perfect result.
- **Analyse the Output**: See what the AI got right and what it got wrong.
- **Refine the Prompt**: Adjust the wording, add more context, clarify instructions, provide examples based on the previous output.
- **Repeat**: Treat prompting as a conversation or iterative process.

Best Practices for Prompt Engineering (2/5): Start Simple, Add Detail

- **Begin with a Core Request**: Start with the main goal of your prompt.
- **Incrementally Add Layers**: Gradually add constraints, context, formatting requirements, persona, examples as needed.
- **Avoid Overwhelming the AI**: Don't try to cram too many complex, unrelated instructions into a single prompt, especially initially.

Best Practices for Prompt Engineering (3/5): Be Explicit & Use Delimiters

- **Leave No Room for Guesswork**: State your requirements clearly and directly. Don't assume the AI knows implicit context or common sense.
- **Use Structural Elements**: Employ delimiters (like ```, ###, <tag></tag>) to clearly separate different parts of your prompt (e.g., instructions, context, examples, input data). This helps the AI parse the prompt correctly.

Best Practices for Prompt Engineering (4/5): Experiment & Specify Steps

- **Try Different Phrasing**: If one prompt doesn't work, rephrase it. Sometimes small wording changes make a big difference.
- **Break Down Complex Tasks**: For very complex requests, break the problem down into smaller steps and prompt the AI for each step sequentially. This is sometimes called "**Chain-of-Thought**" prompting. (e.g., "First, identify the main classes needed. Second, define the attributes for class X. Third, write the constructor for class X...").

Best Practices for Prompt Engineering (5/5): Security and Privacy

- Avoid including **sensitive** or **personal** information in prompts.
- Steer clear of **sensitive** system details.
- Avoid prompts that might lead to **unethical** or **harmful** practices.
- Prompts should embrace **diversity** and **inclusion**.
- Example:
  - "How would you fix a login issue reported by John Doe at john.doe@example.com?" (Bad)
  - "How would you tackle a login issue reported by a user?" (Good)

Key Takeaways

- Introduced Generative AI and its role in code generation via AI Copilots.
- Discussed the significant Benefits and critical Drawbacks/Risks of these tools.
- Defined Prompt Engineering as the crucial skill for interacting effectively with AI.
- Explored Key Elements and Best Practices for crafting high-quality prompts.
- AI Copilots are powerful assistants, but effective and responsible use requires critical thinking and skilled communication (prompting).

*** Please always review and make sure you understand your code.**

AI can be a great copilot, but you are always the pilot!

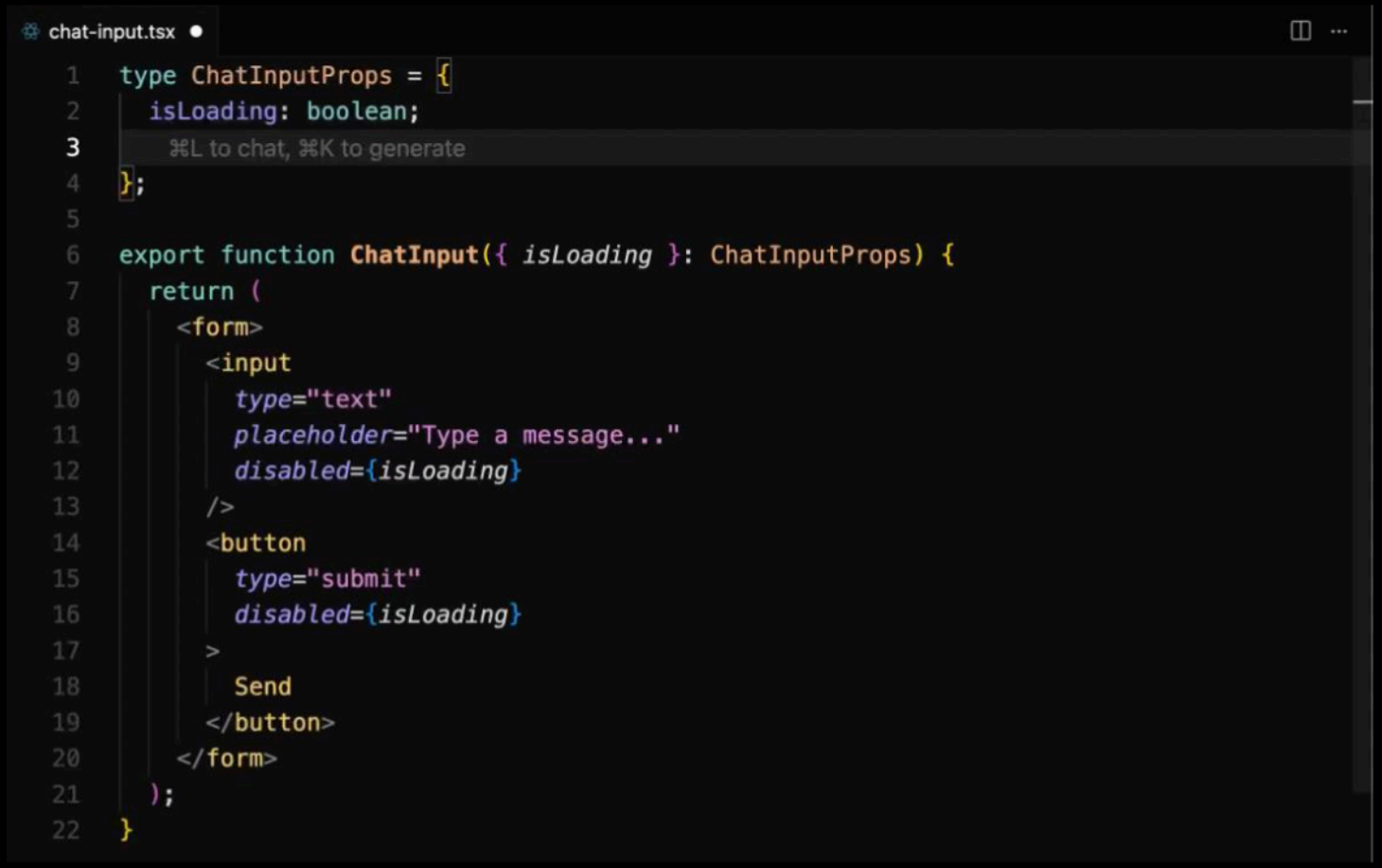
Q & A

VICTORIA UNIVERSITY
MELBOURNE AUSTRALIA

Lab

Setup Your AI Copilot - Cursor

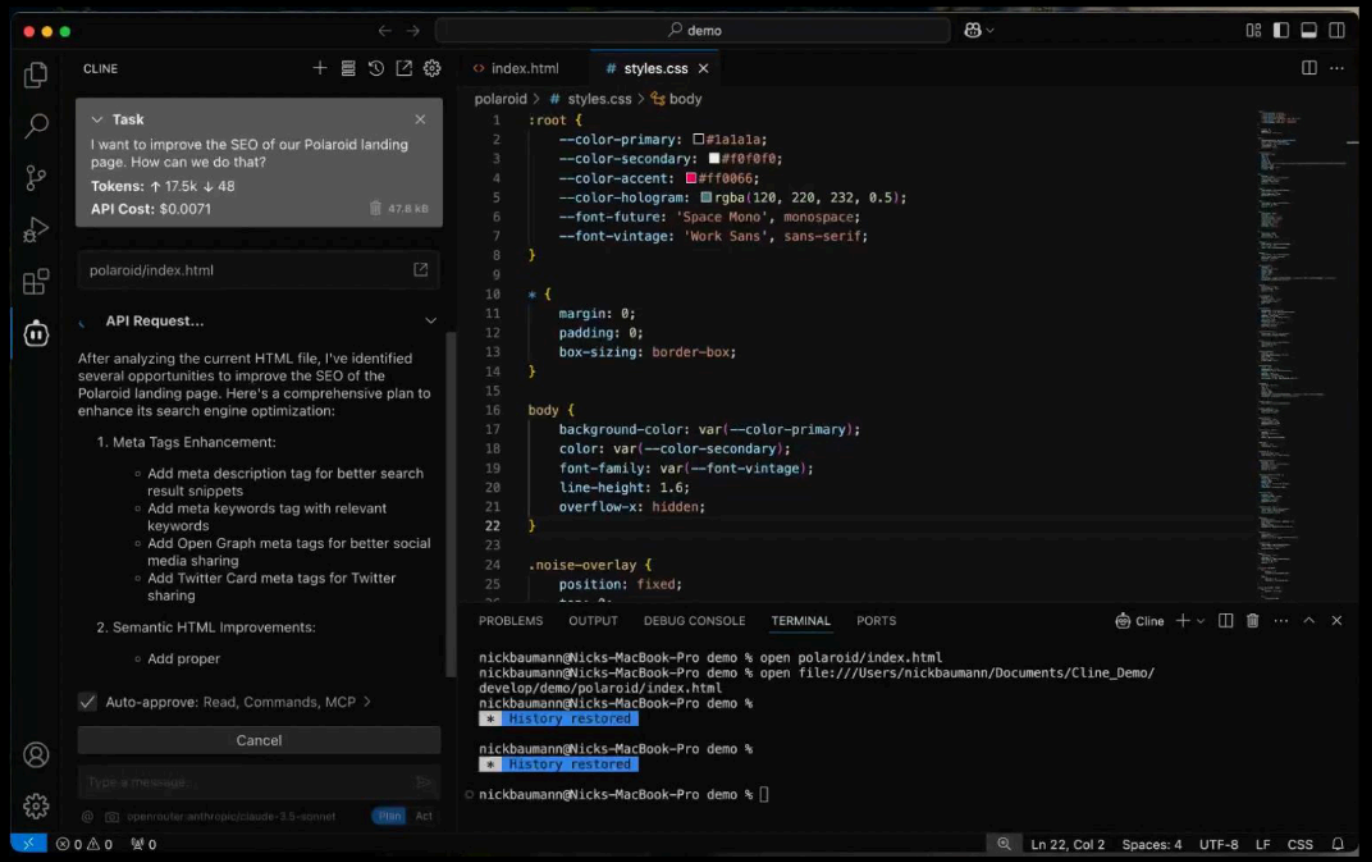
- **Cursor**: Knows your codebase, and Tab, tab, tab. Cursor lets you breeze through changes by predicting your next edit.
- Set up Cursor in your VS Code: https://docs.cursor.com/get-started/installation



https://cursor.com/en

Setup Your AI Copilot - Cline

- **Cline**: open-source AI coding assistant with dual Plan/Act modes, terminal execution, and Model Context Protocol (MCP) for VS Code
- Set up Cline in your VS Code: https://docs.cline.bot/getting-started/for-new-coders
- You may choose your favourite one.
- Alternatively, you can create an account and interact with ChatGPT directly through a web-based chat interface for assistance.

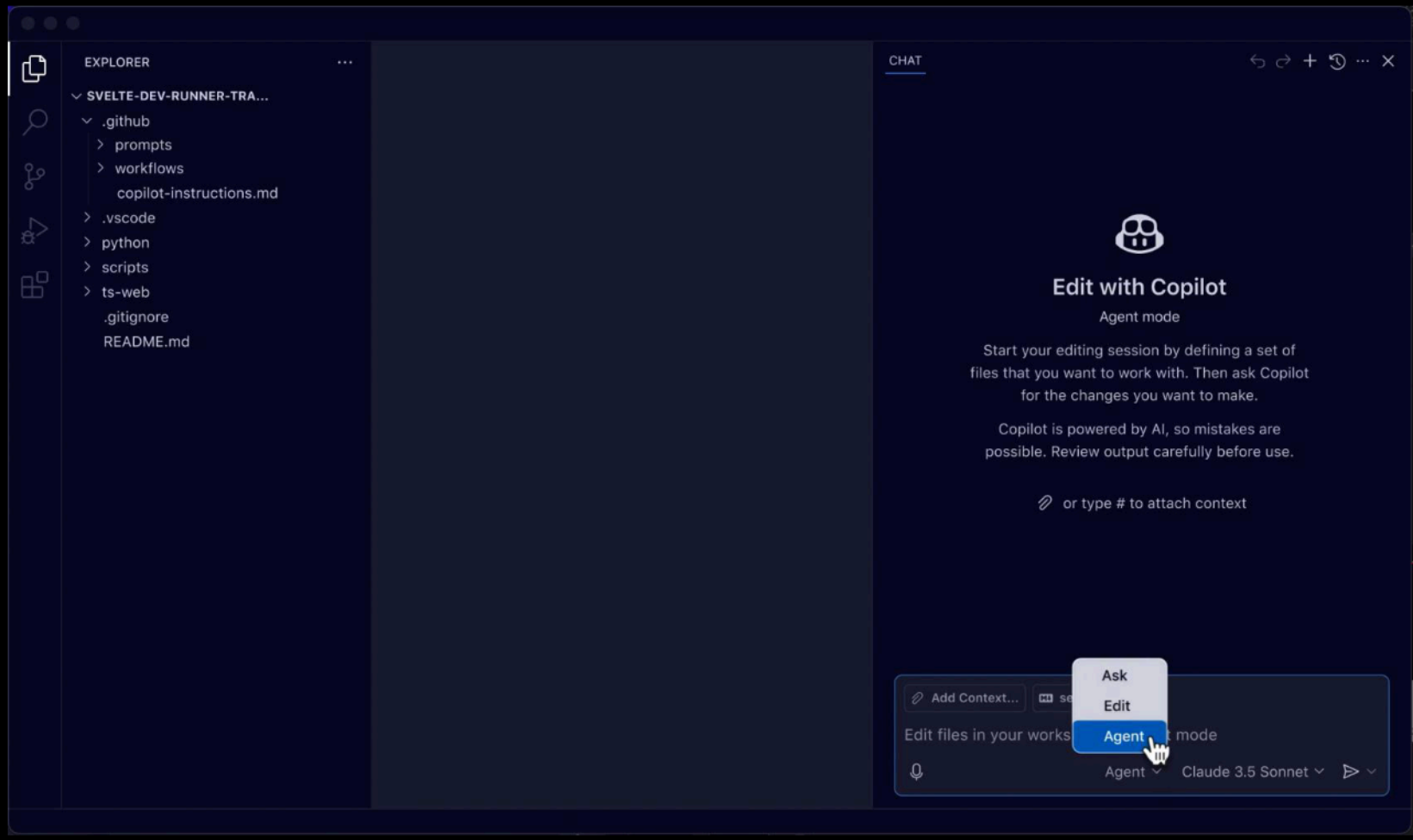


https://cline.bot/

Setup Your AI Copilot - GitHub Copilot

- **GitHub Copilot**: an AI coding assistant that is available on the GitHub website, in GitHub Mobile, in supported IDEs (Visual Studio Code, Visual Studio, JetBrains IDEs, Eclipse IDE, and Xcode), and in Windows Terminal.
- Set up GitHub Copilot in your VS Code: https://code.visualstudio.com/docs/copilot/setup

- You may choose your favourite AI Copilot.
- Alternatively, you can create an account and interact with Chatbot (ChatGPT, Claude, Google Gemini) directly through a web-based chat interface for assistance.



https://code.visualstudio.com/assets/docs/copilot/setup/copilot-chat-view-welcome.png